

Lab 3: Architecture Selection - Justification

Tejas P | SRN: PES1UG24CS497 | Section I

Architecture Selection

We chose Layered Architecture for the Faculty Research Grant & Publication Tracker System.

I split the system into three layers. The Web Portal UI is on top, the business components are in the middle (authentication, expense manager, approval workflow, report generator, publication manager, notification service and the indexing adapter), and the audit ledger and database are at the bottom. The only outside system is the Indexing Service, which we reach through one adapter. The diagram has 10 components and 11 interfaces.

I also looked at the other two styles. Microservices would let every part scale on its own, but this system only expects about 50 users at a time, so the extra complexity is not worth it. Plain client-server is simpler, but it keeps the screens, rules and data access together on one server. That makes it hard to enforce the ledger rules in one place, and the server becomes a single point of failure.

Two Reasons for the Choice

- **1. The money data has to stay correct.** When a researcher logs an expense, the system checks the balance. If the expense is more than what is left, it is held until the Research Dean approves it (FR-004). Only then is it posted to the ledger and the balance updated. With one database in the data layer this can be a single transaction. With microservices the ledger and the balance would sit in different services, and one of them could fail while the other succeeds.
- **2. It fits the size of the project.** NFR-002 only asks for 50 users at once and up to 20 grants per researcher, so we don't need to scale parts separately. One application with clear layers is easier to build and test sprint by sprint, and it lines up with our Lab 2 epics: expenses use the Expense Manager and Approval Workflow, publications use the Publication Manager and Indexing Adapter, and co-author approvals and reports use the Approval Workflow and Report Generator.

Security Advantage

Because of the layers, every request has to go through the same checkpoints. The UI cannot reach the database directly. It has to call the business components, and the Authentication Service checks whether the user is a Faculty Researcher or the Research Dean (only the Dean can approve an overdraft). Every financial change then goes to the Audit Ledger, which is append-only, so a posted entry cannot be edited or deleted (NFR-001). Only the data layer can talk to the database, and the outside Indexing Service stays behind its adapter.

Performance Benefit

All the layers run inside one application, so they call each other directly instead of over the network like microservices would. That helps keep the dashboard loading in under 3 seconds (NFR-002). Data that is read often, like balances, can also be cached in the business layer, and the Indexing Adapter uses a timeout so a slow outside service doesn't freeze the portal (FR-002 wants citation counts within 5 seconds). **Downside:** layers add a little overhead and the whole app scales together, but that is fine at this size.